SPRINT 3 PLAN

Simulation Engine

Project	AI for Equitable Public Transportation — Miami-Dade County	**Client**	Deloitte / UM Herbert Business School
Status	PENDING — Sprint 2b complete	**Date**	April 2, 2026

OVERVIEW

Sprint 3 builds the **simulation engine** that allows planners to ask counterfactual questions about Miami-Dade transit equity: *"If we add 2 peak-hour trips to Critical tracts, how many tier upgrades result?"* or *"If poverty rises 3% in fragile tracts, how much does projected equity worsen?"*

The engine is a direct extension of the two-model architecture developed in Sprint 2b. It uses the **leakage-free XGBoost v3 model** (CV R^2=0.823) for the transit access deficit side, and the **OLS need model** from 2b.6 (R^2=0.82) for the demographic need side. Both sides combine multiplicatively — the same formula used throughout Sprint 2.

Pipeline Architecture

`User Input (GTFS levers)`	→	`XGBoost v3`	→	`updated_service_deficit`
`User Input (demographic levers)`	→	`OLS need model`	→	`updated_projected_need`
		↓ `multiply` ↓		
		`new_equity_score = updated_need × updated_deficit`		
		`new_tier (Sprint 2a fixed cutoffs)`		

Need-side approach: Rather than re-running the full TimeSeries model, the simulator uses a **linear delta method**: OLS coefficients (poverty_rate, no_vehicle) from 2b.6 are stored as a JSON and applied as `new_need = projected_need_2027 + (coef_poverty × Δpoverty) + (coef_no_vehicle × Δno_vehicle)`. Transparent, fast, and appropriate for a planning tool.

Sub-Sprint Summary

#	Sub-Sprint	Outputs
3.1	Baseline State & Lever Catalog	Sprint3_Baseline_State.csv Sprint3_Lever_Catalog.json Sprint3_Need_OLS_Coefficients.json
3.2	simulator.py — Core Engine Module	simulator.py test_simulator.py

3.3	Scenario Validation Notebook	Sprint3_Scenario_Validation.ipynb Sprint3_Scenario_Results.csv
3.4	Interactive Prototype (Gradio)	Sprint_3_Gradio_V3.ipynb (upgrade of existing V2 draft)

3.1 BASELINE STATE & LEVER CATALOG

Build the single source of truth the simulator loads at startup. All 504 tracts, all 13 model features, plus the predicted deficit, projected need 2027, current equity score, tier, and fragile flag — one row per tract, one file.

A. Assemble Baseline State

Inner-join four Sprint 2b outputs on `tract_geoid`. Row count must remain 504 after all joins.

Source File	Columns Used
Sprint2b_Modeling_Features_NotebookOutput.csv	13 scheduling-quality model features (all XGBoost v3 inputs)
Sprint2b_Risk_Scores_v3.csv	deficit_predicted, deficit_actual (XGBoost v3 baseline)
Sprint2b_Combined_Risk_Scoring.csv	projected_need_2027, projected_equity_2027, equity_tier, fragile_flag
Sprint2_Equity_Indicators_v3_tract.csv	Tier cutoff anchors + population per tract

B. Sanity Check — Re-run XGBoost

Load `Sprint2b_XGBoost_v3.pkl` + `Sprint2b_Scaler_v3.pkl`. Re-predict `service_deficit` on the loaded 13-feature matrix. Compare to stored `deficit_predicted`. **Max absolute deviation must be < 1e-4.** If not, flag — likely a feature column ordering or scaling mismatch.

C. Extract OLS Need Coefficients

Open `Sprint2b_Modeling_RiskScoring.ipynb` Section 11. Pull `coef_poverty_rate`, `coef_no_vehicle`, and `intercept`. Store in `Sprint3_Need_OLS_Coefficients.json` with source comment.

D. Lever Catalog

8 adjustable levers across both model sides. Spatial and commute-trend features are explicitly held fixed (non-adjustable).

Feature	Side	Unit	Range	Notes
freq_peak_am_tph	Access	trips/hr	[–current, +10]	SHAP #1 — 39.6% importance. Primary driver.
weekend_weekday_ratio	Access	ratio	[0, 1]	SHAP #2 — 21.7%. Weekend service parity.

freq_early_tph	Access	trips/hr	[–current, +5]	SHAP #3 — 6.9%. Shift-worker coverage.
headway_peak_am_min	Access	minutes	[–20, +20]	SHAP #6 — 3.8%. Inversely related to freq_peak.
headway_early_min	Access	minutes	[–20, +20]	Early-morning headway. Paired with freq_early.
rail_trip_share	Access	fraction	[0, 1]	Modal mix lever. Rail vs. bus balance.
poverty_rate	Need	Δ fraction	[–0.15, +0.15]	Applied to projected_need_2027 via OLS coef.
no_vehicle	Need	Δ fraction	[–0.20, +0.20]	Applied to projected_need_2027 via OLS coef.

Non-adjustable (held fixed): *neighbor_mean_access_score, neighbor_deficit_lag, distance_to_high_service_tract (geographic), trend_commute_* (historical trends).*

3.2 SIMULATOR CORE MODULE (simulator.py)

A clean, importable Python module — no Jupyter dependencies. Sprint 4 imports this directly for dashboard integration.

Public API

```
class Simulator: def __init__(self, baseline_path, model_path, scaler_path,
need_coef_path) def run( self, access_deltas : dict = {}, # {feature: delta}
need_deltas : dict = {}, # {poverty_rate/no_vehicle: delta} tract_filter : list|str =
'all', # geoid list or 'all' label : str = 'scenario' ) -> SimulationResult class
SimulationResult: tract_df : DataFrame # per-tract before/after tier_shifts :
DataFrame # 4x4 from_tier x to_tier matrix summary : dict # n_improved, n_upgrades,
pop_affected, avg_delta label : str
```

Internal Pipeline

Step	Method	Logic
1	_apply_access_delta()	Copy feature matrix. Apply deltas to target tracts only. Clamp to valid ranges from lever catalog (no negative freq, no ratio > 1).
2	_predict_deficit()	scaler.transform(features) → xgb.predict() → clamp to [0, 1]. Column order must match training order exactly.
3	_apply_need_delta()	new_need = projected_need_2027 + coef_poverty × Δpoverty + coef_no_vehicle × Δno_vehicle → clamp to [0, 1].
4	_compute_equity()	need × deficit → assign tier using Sprint 2a fixed cutoffs (not re-quantiled).

5	_diff_report()	Per-tract deltas, 4×4 tier shift matrix, population impact summary.

Validation Gates (must all pass before any scenario runs)

Gate	Assertion
Zero-delta test	run() with no deltas. Per-tract equity must match baseline within 1e-5. Catches model loading or scaling errors.
Isolation test	Apply delta to a tract subset. Unaffected tracts must be bit-for-bit identical to baseline. Catches indexing bugs.
Direction test — access	Increase freq_peak_am_tph → avg service_deficit must fall. Consistent with SHAP #1 driver (39.6% importance).
Direction test — need	Increase poverty_rate → avg projected_need must rise. Consistent with OLS R^2=0.82.
Formula test	Increase need with fixed deficit → avg equity must rise. Validates the need × deficit multiplication.

test_simulator.py — all five validation gates are encoded as pytest assertions and must pass cleanly before 3.3 can begin.

3.3 SCENARIO VALIDATION NOTEBOOK

Five reference scenarios run through the simulator. Results are the Sprint 3 Deloitte deliverable — demonstrating both model sides, the decomposition capability, and magnitude consistency with SHAP rankings.

Reference Scenarios

#	Name	Side	Lever	Scope	Expected Outcome
S1	Peak Frequency Boost	Access	freq_peak_am_tph +2 trips/hr	53 Critical tracts	Deficit drops; some tier upgrades to High. Largest effect (SHAP #1).
S2	Weekend Parity County-Wide	Access	weekend_weekday_ratio → 0.85	All 504 tracts	Tracts with low current ratio improve most (SHAP #2).
S3	Early Service Expansion	Access	freq_early_tph +2 trips/hr	High + Critical (~157)	Shift-worker tracts benefit. Smaller effect than S1 (SHAP #3).
S4	Poverty Stress Test	Need	poverty_rate +0.03	Fragile-flagged tracts	Need rises, equity worsens, tier downgrades expected.

S5	Combined Intervention	Both	S1 access + poverty +0.02 (Moderate)	Mixed	Decomposed output: access delta vs. need delta vs. net equity change.

Per-Scenario Report Requirements

- Per-tract before/after table: deficit, need, equity score, tier
- 4×4 tier shift matrix (from_tier × to_tier counts)
- Summary: n tracts improved, n tier upgrades/downgrades, population affected, avg equity delta
- Magnitude sanity check: S1 effect > S3 effect (39.6% vs 6.9% SHAP importance)
- S5 decomposition: access-side delta and need-side delta shown separately, then combined net

Pass Criteria

Criterion	Condition
Validation gates	All 5 gates in test_simulator.py pass (pytest)
SHAP consistency	S1 > S3 per-tract deficit delta (freq_peak 39.6% vs freq_early 6.9%)
Need linearity	S4 need response consistent with OLS R^2=0.82 (tight relationship)
S5 bounds	Combined result falls between S1-only and S4-only bounds
Score validity	No equity score outside [0, 1]; no tier outside {Critical, High, Moderate, Low}

3.4 INTERACTIVE PROTOTYPE (Gradio)

An existing Gradio draft (`Sprint_3_Gradio_V2.ipynb`) is already in the Sprint 3 folder and partially functional. This sub-sprint upgrades it to plug into `simulator.py` and produce a working before/after demo. This is **not** the Sprint 4 production dashboard — it is a pre-dashboard proof-of-concept for internal review and Deloitte demos.

What V2 Already Has (keep as-is)

Feature	Description
Choropleth map	504 tracts colored by risk tier; tier filter + score threshold slider
Tract narrative	Enter a GEOID → plain-English transit situation summary
Summary stats panel	Tier distribution, model R^2, county-level overview
Geometry	Auto-fetched from Census TIGER API — no shapefile needed
Model loading	Loads Sprint2b_XGBoost_v3.pkl via joblib; falls back to composite_access_deficit proxy if pkl missing

Current Limitation — What-If Simulator

V2's what-if panel simulates a **single synthetic tract**: it fills in median values for 9 of the 13 features and exposes only 4 sliders. It does not run against the 504-tract baseline, does not use `simulator.py`, and produces no before/after map or tier shift output. This is the only thing that needs fixing.

Upgrades for V3

	Upgrade	Details
A	Wire what-if panel to simulator.py	Replace median-filling logic with Simulator.run(). Sliders: freq_peak_am_tph, headway_peak_am_min, weekend_weekday_ratio, rail_share, freq_early_tph, poverty_rate delta, no_vehicle delta. Add tract scope selector: All / Critical only / High+Critical / custom GEOID list.
B	Side-by-side before/after map	Two choropleth panels (current vs. scenario) using the same TIGER geometry. Tracts that changed tier are visually highlighted. Keep it simple — two maps, no animation.
C	Scenario summary panel	Below the maps: tier shift matrix, n tracts improved, population affected, avg equity delta. Pulled directly from SimulationResult.tier_shifts and SimulationResult.summary.
D	Keep V2 features unchanged	Map filter, narrative generator, summary stats panel are good — do not touch them.

Dependency & Scope Boundaries

• **Hard dependency:** 3.4 cannot start until 3.2 (simulator.py) is complete. 3.3 and 3.4 can run in parallel once 3.2 is done.

• **Not in scope:** trend projections, fragile tract alerts, export capability, time-series animation — all owned by Sprint 4.

• **Not the production dashboard:** Sprint 4 owns the Plotly Dash / Streamlit build. This Gradio prototype is for demos and design validation only.

REFERENCE FILES FROM SPRINT 2

File	Location	Used For
Sprint2b_XGBoost_v3.pkl	Sprint 2 /	Trained XGBoost access deficit model (3.1 sanity check, 3.2 engine)
Sprint2b_Scaler_v3.pkl	Sprint 2 /	Feature scaler matched to XGBoost training
Sprint2b_Modeling_Features_NotebookOutput.csv	Sprint 2 /	504 tracts × 61 cols — source of 13 model features for baseline
Sprint2b_Risk_Scores_v3.csv	Sprint 2 /	Per-tract deficit_predicted and deficit_actual (XGBoost v3)

Sprint2b_Combined_Risk_Scoring.csv	Sprint 2 /	projected_need_2027, equity_tier, fragile_flag — need-side baseline
Sprint2b_Modeling_RiskScoring.ipynb	Sprint 2 /	Section 11: OLS need model coefficients (poverty, no_vehicle, intercept)
Sprint2_Equity_Indicators_v3_tract.csv	Sprint 2 /Composite Equity Indicators/	Tier cutoff anchors + population per tract

SPRINT 3 OUTPUTS

File	Sub-Sprint	Description
Sprint3_Baseline_State.csv	3.1	504 tracts: 13 features + deficit + need + equity + tier + fragile flag
Sprint3_Lever_Catalog.json	3.1	8 levers with feature names, units, valid ranges, SHAP notes
Sprint3_Need_OLS_Coefficients.json	3.1	OLS coefficients for need-side delta computation
simulator.py	3.2	Importable simulation engine (Sprint 4 dependency)
test_simulator.py	3.2	Pytest suite — 5 validation gates
Sprint3_Scenario_Validation.ipynb	3.3	Scenario narrative notebook (Deloitte deliverable)
Sprint3_Scenario_Results.csv	3.3	504 tracts × 5 scenario columns: equity_delta, tier_before, tier_after
Sprint_3_Gradio_V3.ipynb	3.4	Interactive Gradio prototype — before/after maps wired to simulator.py

All outputs saved to `Sprint 3 /` *folder.* `simulator.py` *is the primary Sprint 4 dependency and also the hard prerequisite for 3.4 — all validation gates must pass before dashboard integration or the Gradio prototype upgrade begins.*